

Qualcomm's BREW was based on C/C++. On the other hand, J2ME (Java 2 Micro-Edition) was a stripped down version of Java's Standard edition, with a limited API set meant for devices with minimal hardware. It was mostly used to build lightweight games, and basic applications for feature phones with limited functions, like those related to PIM (Personal Information Management). J2ME devices hardly allowed anything related to system/hardware interfacing and even if they did, things rarely worked as intended. This limited J2ME to be used mostly for gaming applications on feature phones and not for any sophisticated client application.

The challenges - poor hardware, as well as limited APIs and multi-tasking: The early feature phones came with quite a few challenges.

1. Building games or apps on J2ME was challenging owing to limited hardware capabilities. Many of the phones allowed a maximum of 64KB of JAR (J2ME binary size) and/or allowed 128KB of RAM. Even the most popular J2ME device in 2005, the Nokia 6600, supported 1MB of RAM for J2ME apps.
2. Also, since the API set was very limited, it only enabled the creation of very basic applications.
3. Multi-tasking was missing on most of the feature phones. J2ME apps used to pause as soon as they were moved to the background.

Alternative approaches to deal with small RAM and app binary sizes: Over the years, the design and development strategies adopted to overcome different challenges included the following.

1. Apps were packaged with very few resources (images), while other required resources were fetched once from the server via data calls and then cached locally to the record management system.
2. Apps were designed to load only a portion of the data or resource (image) required at any moment of time. Later, loaded resources were replaced with other resources needed for subsequent flows. For example, for one of the gaming applications (Shadow Showdown's FOP), we chopped the big image of a character animation into three independent parts to keep the memory in check by optimising the size of the image part. We then used one part of the image at a time and then replaced it with another part when needed.
3. Usually, basic apps were built without a rich API set and no multi-tasking.

2005-09: The BlackBerry era heralds the first smart, business phone

Roughly between 2005 and 2010 was when BlackBerry phones enjoyed the most popularity. BlackBerry from RIM (Research in Motion) was the first smartphone introduced in the league of phones/platforms discussed in this article. Because of its security, it was considered the best business phone and preferred by enterprises. BlackBerry had its own OS, which provided multi-tasking and initially supported devices with a trackwheel

and trackpad. The brand had many popular series of phones, like Curve, Pearl and Storm, to name a few.

BlackBerry provided a Java based API set. With rich APIs and multi-tasking supported by the OS, it became possible for developers to build sophisticated native business apps which hadn't been the case for J2ME apps. Being able to use built-in device capabilities allowed apps to have features like file operations, native email, phone and SMS.

Rich applications, right from Reuter's stock market app and the Stitcher radio (audio) app to OpenTable's restaurant booking app and Broadworks Assistant VoIP calling background app, were all possible with RIM. Yet, creating mobile websites or Web based applications wasn't feasible owing to very limited JavaScript support.

The challenges: Some of the main shortcomings of the BlackBerry included a small display, a complex UI API, an old Web kit and difficult backward compatibility.

1. One of the biggest challenges for BlackBerry's popular 4.x device series (that included Curve and Pearl) was designing the UX and implementing it. BlackBerry phones already had only a single column view compared to screens that permitted the multi-column view for a wider Web area. On top of this, the hard keypad hindered the possibility of increasing the display height effectively.
2. The only way to create a user interface (UI) for BlackBerry's Java apps was by using RIM native APIs. The UI API set and layout principles were complex, and this made it difficult to develop a rich UI.
3. The majority of BlackBerry users in the market had the old RIM OS version (4.x series) running on their devices, and the Web kit engine that came with 4.x was limited in terms of functionality and JavaScript capabilities. Hence, building mobile websites became really difficult. I remember that building an interactive Web calendar control became almost impossible for the BlackBerry OS 4.x series.
4. RIM supported static class verification, that is, if an app was compiled and packaged with a higher OS version's native library API suite and possessed a higher OS API, then the packaged app would not install on devices with old OS versions. This made it difficult to enable backward compatibility while creating apps targeted for latest OS versions.

Dealing with the challenges: Listed below are some of the design and development strategies used to overcome the different challenges.

1. All key modules were served as *Home Screen* options, after which standard linear or tabbed (for big apps) navigation followed.
2. A reusable custom compounded and UI component library was created. This was done by extending standard containers and components (like a field manager, vertical/horizontal field managers, label/text fields, etc).
3. As 4.x devices covered a majority of the market initially, native RIM applications were the only option until the